

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyse DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships. (BL1, BL2)*

Activity Tool : Concept Mapping (Medium)

Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Construct a concept map linking the following terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship.	BL2 – Understand
2	Create a concept map for the three-level ANSI/SPARC architecture showing data independence at each level.	BL2 – Understand

3	Map the relationship between file-based systems and database systems, highlighting the problems solved by DBMS.	BL2 – Understand
4	Design a concept map for ER diagram components: entity, weak entity, attribute types, relationship types, and participation constraints.	BL2 – Understand
5	Construct a concept map comparing Hierarchical, Network, and Relational data models with their key properties.	BL2 – Understand
6	Develop a concept map for the EER model showing how it extends the basic ER model with generalization, specialization, and aggregation.	BL2 – Understand
7	Create a visual concept map for the DBMS software architecture, illustrating the flow from user request to data retrieval.	BL2 – Understand
8	Map the data models discussed in CO1 to real-world applications and explain the suitability of each model.	BL2 – Understand
9	Construct a concept map for schemas and instances, showing how they relate to physical and logical data independence.	BL2 – Understand
10	Design a concept map for the roles and responsibilities in a DBMS environment: DBA, End User, Application Developer.	BL1 – Remember

Code of Conduct:

I A.Gunavathi(192511374) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

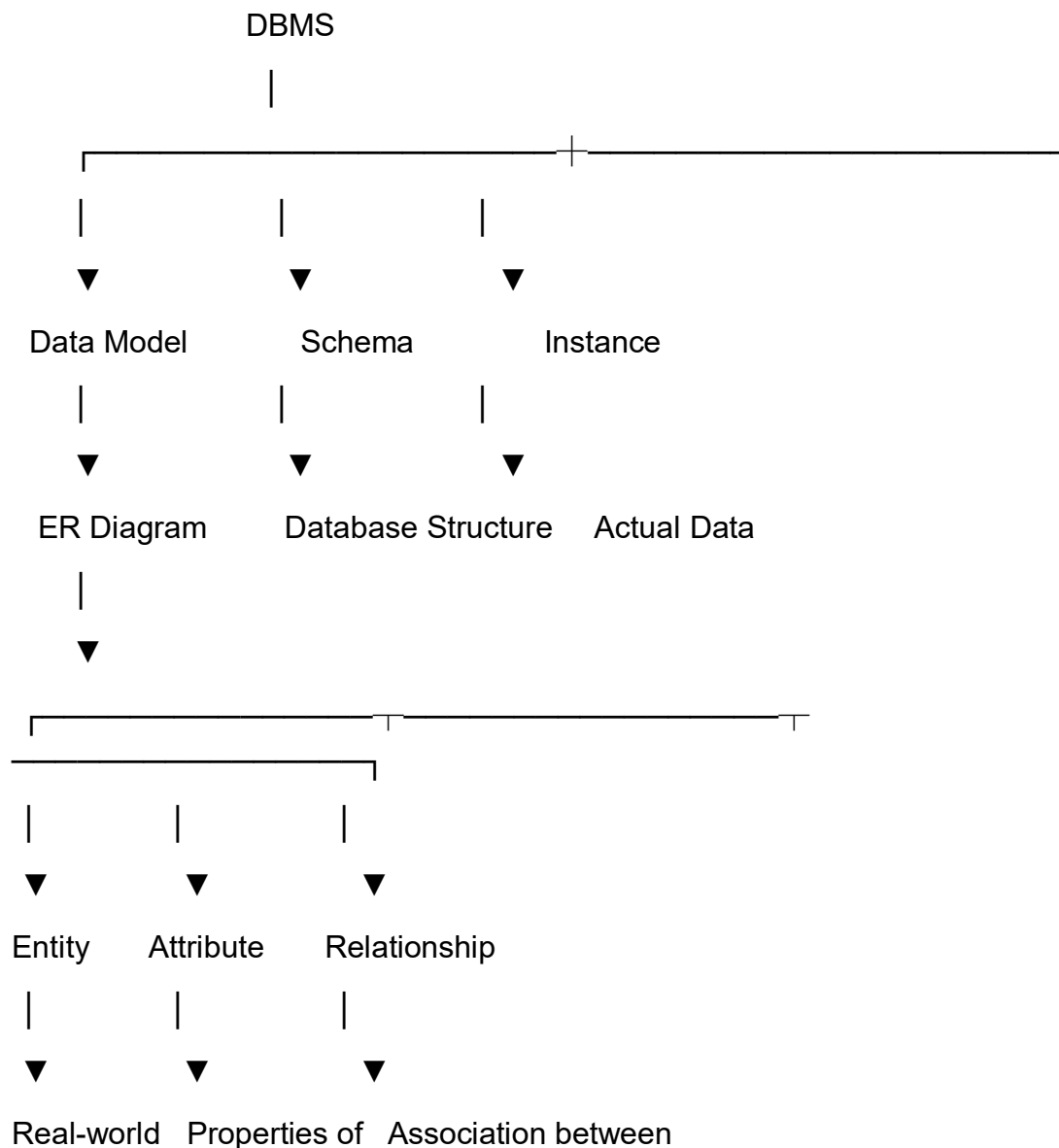
Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence

Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand
---------	-----	--	---	---	---

Q1. Construct a Concept Map Linking the Following Terms: DBMS, Schema, Instance, Data Model, ER Diagram, Entity, Attribute, Relationship



Object an Entity Entities

Introduction

A **Database Management System (DBMS)** is software used to create, store, retrieve, and manage data efficiently. To design and organize a database, concepts such as **Data Model, ER Diagram, Entity, Attribute, Relationship, Schema, and Instance** are used. These concepts are interconnected and form the foundation of database design.

Explanation of Terms

1. DBMS

A **DBMS (Database Management System)** is software that manages databases by storing, retrieving, updating, and securing data.

Examples: MySQL, Oracle, SQL Server.

2. Data Model

A **Data Model** defines how data is organized, stored, and related in a database.

Examples:

- Hierarchical Model
 - Network Model
 - Relational Model
 - ER Model
-

3. ER Diagram

An **Entity-Relationship (ER) Diagram** is a graphical representation of the database that shows entities, attributes, and relationships.

4. Entity

An **Entity** is a real-world object or person about which information is stored.

Examples: Student, Employee, Book.

5. Attribute

An **Attribute** describes the properties of an entity.

Example:

Entity: **Student**

Attributes:

- Student_ID
 - Name
 - Department
-

6. Relationship

A **Relationship** is an association between two or more entities.

Example:

Student **Enrolls** in Course.

7. Schema

A **Schema** is the logical structure or blueprint of a database. It defines tables, attributes, keys, and relationships.

Example:

Student(Student_ID, Name, Department)

8. Instance

An **Instance** is the actual data stored in the database at a particular point in time.

Example:

Student_ID	Name	Department
------------	------	------------

101	Rahul	CSE
-----	-------	-----

102	Priya	AI
-----	-------	----

Relationship Among the Terms

- **DBMS** manages the database.
- A **Data Model** defines how the database is designed.

- An **ER Diagram** is created using the data model.
- The ER Diagram consists of **Entities**, **Attributes**, and **Relationships**.
- These are converted into a **Schema** (database structure).
- The **Instance** contains the actual data stored according to the schema.

Summary Table

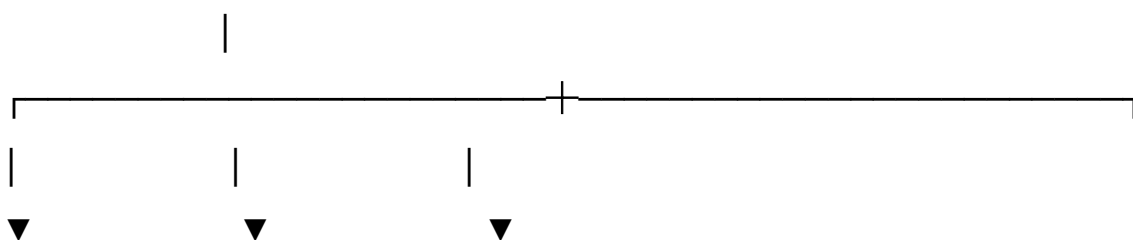
Term	Description
DBMS	Software that manages databases
Data Model	Defines database structure
ER Diagram	Graphical representation of a database
Entity	Real-world object
Attribute	Property of an entity
Relationship	Association between entities
Schema	Logical structure of the database
Instance	Actual data stored in the database

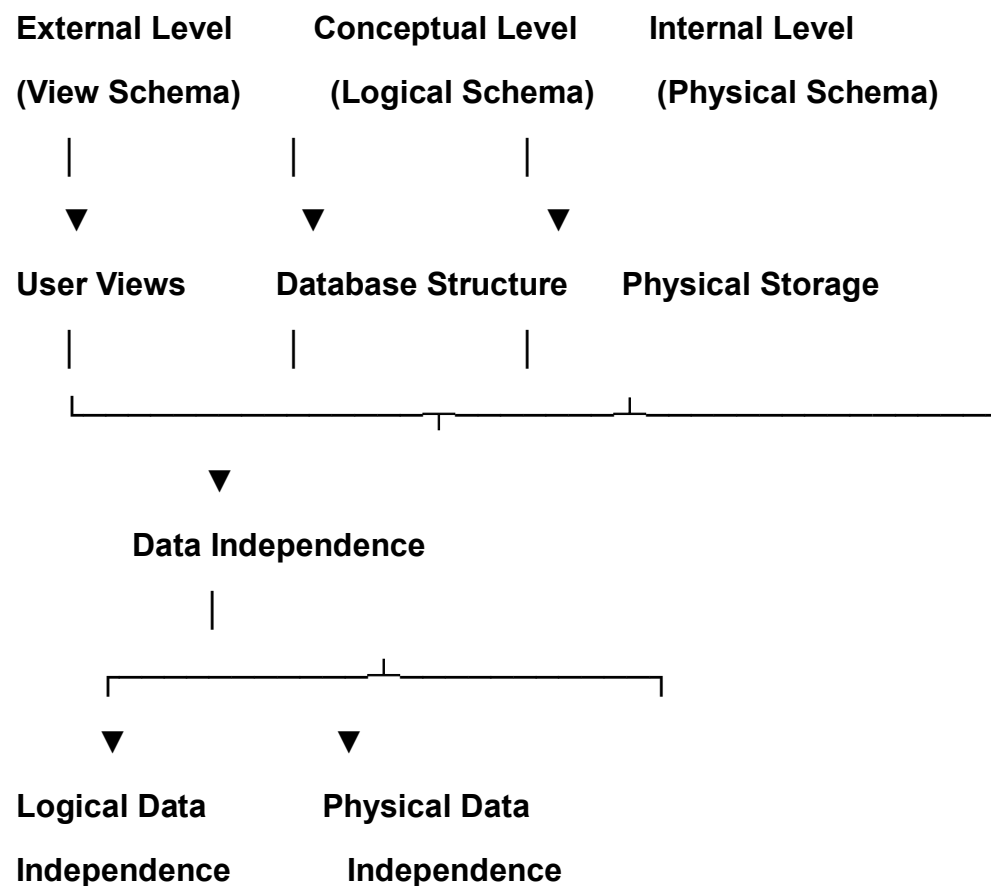
Conclusion

The **DBMS** is the software that manages databases efficiently. A **Data Model** provides the structure for database design, while an **ER Diagram** represents entities, attributes, and relationships visually. These concepts are used to create the **Schema**, and the **Instance** stores the actual data. Together, they form the foundation of an efficient database system.

Q2. Create a Concept Map for the Three-Level ANSI/SPARC Architecture Showing Data Independence at Each Level.

ANSI/SPARC Architecture





Introduction

The ANSI/SPARC Three-Level Architecture is a standard database architecture used in Database Management Systems (DBMS). It divides a database into three levels: External Level, Conceptual Level, and Internal Level. This separation provides Data Independence, allowing changes at one level without affecting the other levels.

Three Levels of ANSI/SPARC Architecture

1. External Level (View Level)

The External Level provides different views of the database to different users.

Features:

- User-specific view.
- Hides unnecessary data.
- Improves security.
- Supports multiple user views.

Example: Students can view only their marks, while faculty can access student records.

2. Conceptual Level (Logical Level)

The Conceptual Level describes the complete logical structure of the database.

It includes:

- **Entities**
- **Attributes**
- **Relationships**
- **Constraints**

Example: A college database contains Student, Faculty, Course, and Department information.

3. Internal Level (Physical Level)

The Internal Level describes how data is physically stored in the computer.

It includes:

- **Files**
- **Indexes**
- **Storage devices**
- **Record organization**

Example: Student records are stored in disk files using indexes.

Data Independence

Data Independence is the ability to change the database schema at one level without affecting the higher levels.

Logical Data Independence

- **Changes in the Conceptual Schema do not affect the External Schema.**
- **Example: Adding a new column to a table without changing user views.**

Physical Data Independence

- **Changes in the Internal Schema do not affect the Conceptual Schema.**

- **Example: Creating indexes or changing file organization without affecting applications.**
-

Advantages

- **Provides data independence.**
 - **Improves database security.**
 - **Supports multiple user views.**
 - **Reduces application maintenance.**
 - **Improves database performance.**
 - **Makes database management easier.**
-

Summary Table

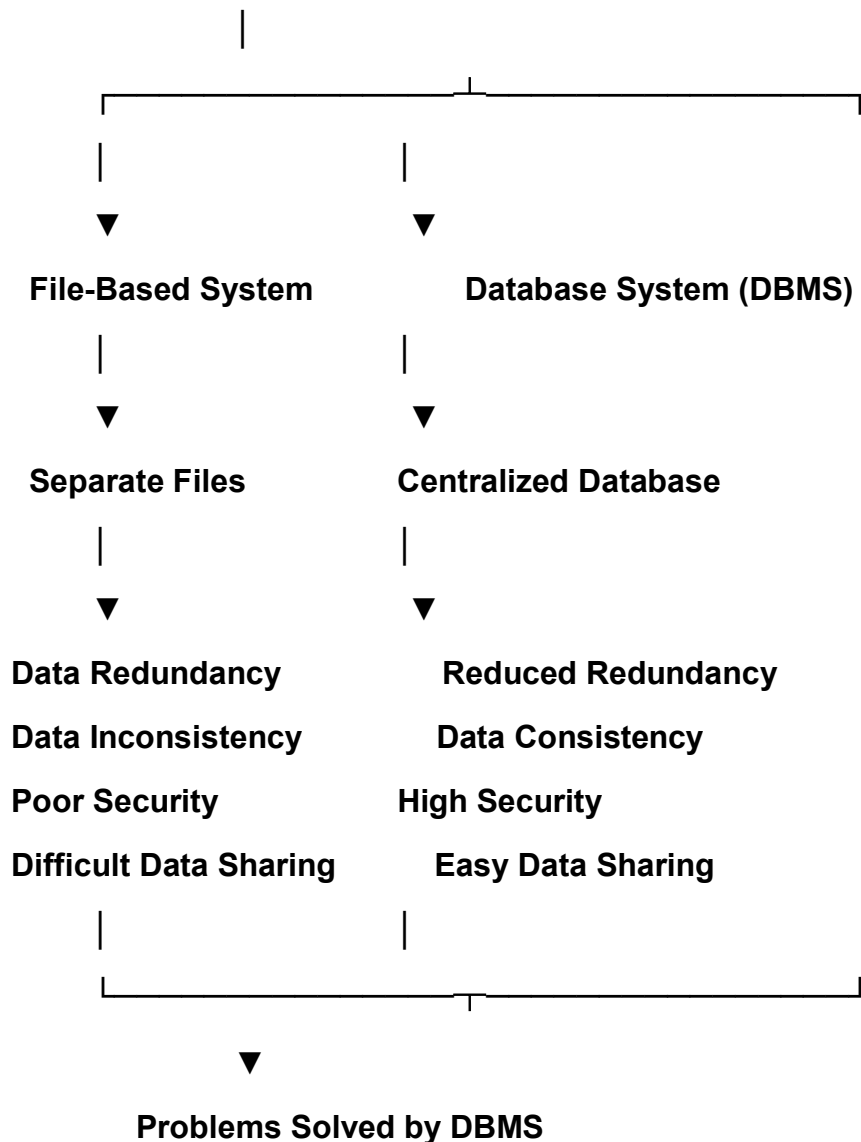
Level	Purpose
External Level	Provides user-specific views
Conceptual Level	Defines the logical structure of the database
Internal Level	Manages physical storage of data
Logical Data Independence	Changes at conceptual level do not affect external level
Physical Data Independence	Changes at internal level do not affect conceptual level

Conclusion

The ANSI/SPARC Three-Level Architecture separates the database into External, Conceptual, and Internal Levels, making database management more flexible and efficient. It provides Logical Data Independence and Physical Data Independence, allowing changes to the database without affecting users or application programs. This architecture improves security, scalability, and maintainability of modern database systems.

Q3. Map the Relationship Between File-Based Systems and Database Systems, Highlighting the Problems Solved by DBMS.

Data Management Systems



Introduction

A File-Based System is the traditional method of storing data in separate files, where each application maintains its own data. This approach often leads to problems such as data redundancy, inconsistency, and poor security.

A Database Management System (DBMS) overcomes these limitations by storing data in a centralized database, allowing efficient storage, retrieval, security, and management of data.

File-Based System

A File-Based System stores data in individual files without a central database.

Features

- Data stored in separate files.

- No centralized control.
- Difficult to share data.
- High data duplication.

Example

A college stores student details, library records, and fee details in separate files.

Database System (DBMS)

A DBMS is software that stores and manages data in a centralized database.

Features

- Centralized storage.
- Data sharing.
- Reduced redundancy.
- Better security.
- Backup and recovery.
- Multi-user support.

Examples: MySQL, Oracle, SQL Server.

Problems in File-Based Systems

- **Data Redundancy:** Same data stored in multiple files.
- **Data Inconsistency:** Different files may contain different values.
- **Poor Security:** Limited protection for data.
- **Data Isolation:** Data scattered across multiple files.
- **No Backup and Recovery:** Difficult to recover lost data.
- **Program-Data Dependence:** Changes in file structure affect application programs.

Problems Solved by DBMS

- Reduces data redundancy.
- Maintains data consistency.

- Provides strong security.
 - Supports easy data sharing.
 - Offers backup and recovery.
 - Ensures data independence.
 - Allows multiple users to access data simultaneously.
-

Comparison Table

File-Based System	Database System (DBMS)
Separate files	Centralized database
High data redundancy	Low data redundancy
Data inconsistency	Data consistency
Limited security	Strong security
Difficult data sharing	Easy data sharing
No backup and recovery	Backup and recovery available

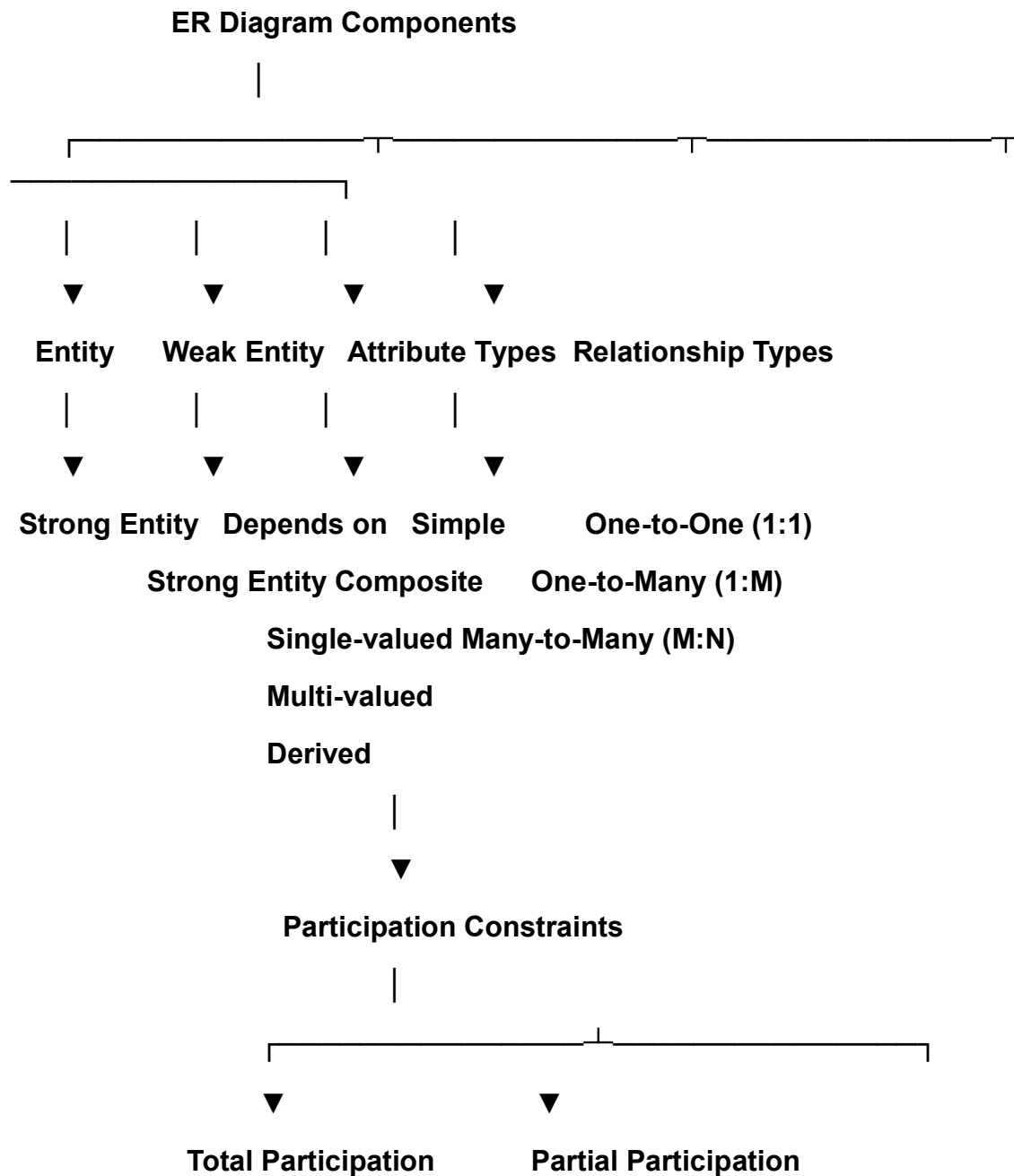
Advantages of DBMS

- Reduces data redundancy.
 - Improves data consistency.
 - Enhances security.
 - Supports concurrent access.
 - Provides backup and recovery.
 - Makes data retrieval faster and easier.
-

Conclusion

A File-Based System has several limitations, including data redundancy, inconsistency, poor security, and lack of data sharing. A Database Management System (DBMS) solves these problems by providing centralized data storage, improved security, data consistency, backup and recovery, and efficient data management. Therefore, DBMS is a more reliable and efficient solution for modern applications.

Q4. Design a Concept Map for ER Diagram Components: Entity, Weak Entity, Attribute Types, Relationship Types, and Participation Constraints.



Introduction

An Entity-Relationship (ER) Diagram is a graphical representation of a database used during database design. It helps in identifying entities, attributes, relationships, weak entities, and participation constraints. These components describe how data is organized and related in a database.

1. Entity

An Entity is a real-world object or person about which information is stored in a database.

Examples

- **Student**
- **Employee**
- **Book**

Representation: Single Rectangle

2. Weak Entity

A Weak Entity is an entity that cannot exist independently. It depends on a Strong Entity for its identification.

Examples

- **Dependent (depends on Employee)**
- **Order Item (depends on Order)**

Representation: Double Rectangle

3. Attribute Types

Attributes describe the properties of an entity.

Types of Attributes

- **Simple Attribute: Cannot be divided further (Age).**
 - **Composite Attribute: Can be divided into smaller parts (Name → First Name, Last Name).**
 - **Single-Valued Attribute: Has only one value (Roll Number).**
 - **Multi-Valued Attribute: Has multiple values (Phone Numbers).**
 - **Derived Attribute: Calculated from another attribute (Age from Date of Birth).**
-

4. Relationship Types

A Relationship represents an association between two or more entities.

Types

- One-to-One (1:1): One person has one passport.
 - One-to-Many (1:M): One department has many employees.
 - Many-to-Many (M:N): Many students enroll in many courses.
-

5. Participation Constraints

Participation constraints indicate whether an entity's participation in a relationship is mandatory or optional.

Total Participation

- Every entity must participate in the relationship.
- Representation: Double line.

Example: Every employee must belong to a department.

Partial Participation

- Participation is optional.
- Representation: Single line.

Example: A customer may or may not place an order.

Example ER Diagram

STUDENT

Student_ID (PK)

Name

Age

|

Enrolls (M:N)

|

▼

COURSE

Course_ID (PK)

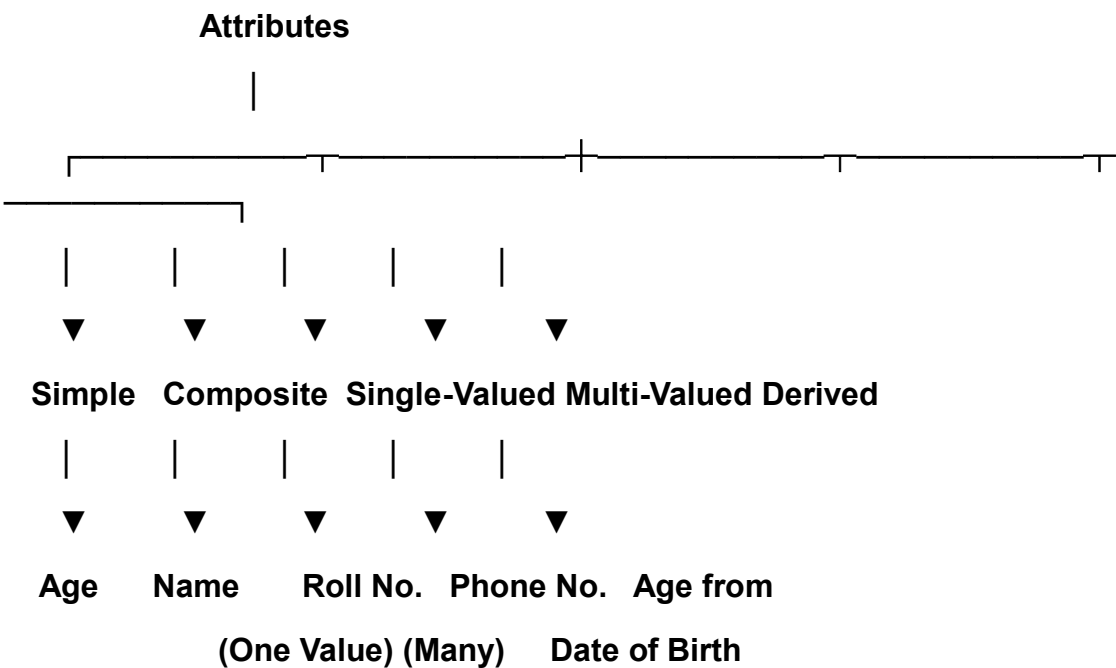
Summary Table

Component	Description
Entity	Real-world object
Weak Entity	Depends on a strong entity
Attribute Types	Describe properties of entities
Relationship Types	Show associations between entities
Participation Constraints	Define mandatory or optional participation

Conclusion

The ER Diagram is an essential tool in database design. It includes entities, weak entities, attribute types, relationship types, and participation constraints to represent the structure of a database clearly. Understanding these components helps in creating efficient, organized, and reliable database systems.

Q5. Create a Concept Map Illustrating the Classification of Attributes (Simple, Composite, Single-Valued, Multi-Valued, Derived) with Examples



Introduction

An Attribute is a property or characteristic of an Entity in an Entity-Relationship (ER) model. Attributes provide detailed information about an entity. Based on their characteristics, attributes are classified into Simple, Composite, Single-Valued, Multi-Valued, and Derived Attributes.

Types of Attributes

1. Simple Attribute

A Simple Attribute cannot be divided into smaller parts.

Example

- **Age**
- **Gender**
- **Salary**

Example: Student → Age

2. Composite Attribute

A Composite Attribute can be divided into smaller sub-attributes.

Example

Name

- **First Name**
- **Middle Name**
- **Last Name**

Address

- **Street**
 - **City**
 - **State**
 - **PIN Code**
-

3. Single-Valued Attribute

A Single-Valued Attribute has only one value for each entity.

Example

- Roll Number
- Aadhaar Number
- Employee ID

Each student has only one Roll Number.

4. Multi-Valued Attribute

A Multi-Valued Attribute can have more than one value for an entity.

Example

- Phone Numbers
- Email IDs
- Skills

A student may have multiple phone numbers.

5. Derived Attribute

A Derived Attribute is calculated from another attribute.

Example

- Age is calculated from Date of Birth.
- Total Salary is calculated from Basic Pay + Allowances.

Derived attributes are generally not stored directly in the database.

Example ER Representation

STUDENT

Roll No.

Name

Age

Date of Birth

Phone Number

- Roll No. → Single-Valued Attribute

- Name → Composite Attribute
- Age → Derived Attribute
- Date of Birth → Simple Attribute
- Phone Number → Multi-Valued Attribute

Advantages of Attribute Classification

- Helps in designing efficient databases.
- Reduces data redundancy.
- Improves data organization.
- Makes database queries easier.
- Ensures better data consistency.

Summary Table

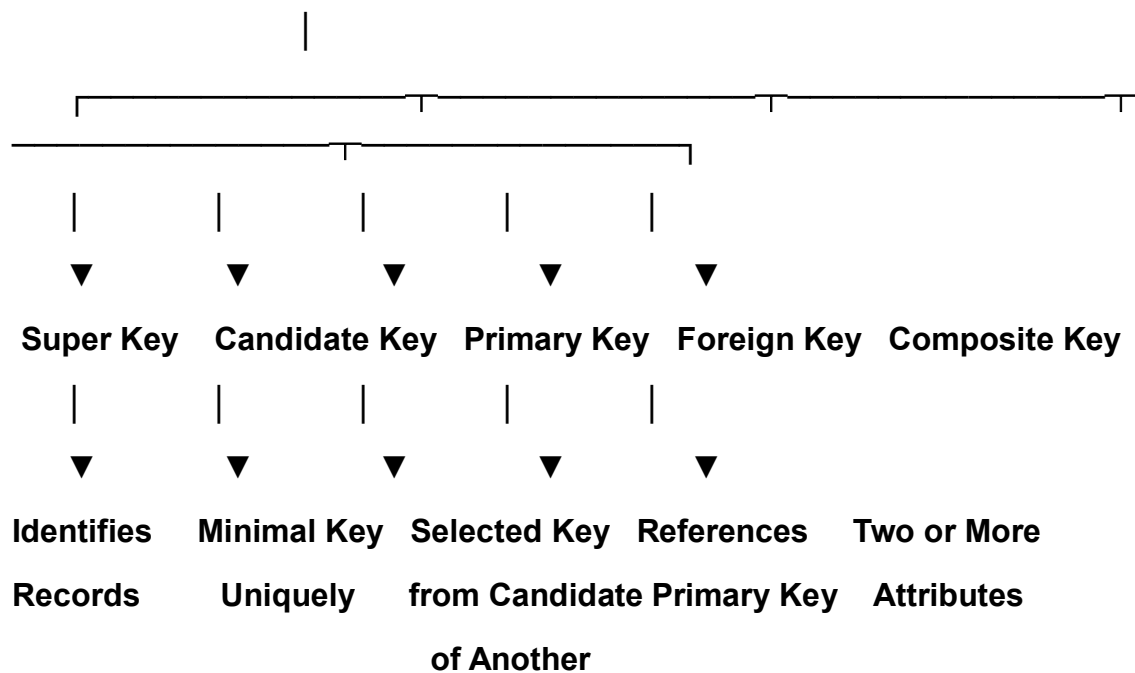
Attribute Type Description		Example
Simple	Cannot be divided further	Age
Composite	Can be divided into sub-parts	Name, Address
Single-Valued	Has only one value	Roll Number
Multi-Valued	Has multiple values	Phone Numbers
Derived	Calculated from another attribute	Age from Date of Birth

Conclusion

Attributes describe the properties of an entity in an ER model. They are classified into Simple, Composite, Single-Valued, Multi-Valued, and Derived Attributes based on their characteristics. Proper classification of attributes helps in creating an efficient database, reduces redundancy, and improves data organization.

Q6. Construct a Concept Map Showing the Relationship Between Keys in a Relational Database (Primary Key, Candidate Key, Super Key, Foreign Key, Composite Key).

Keys in Relational Database



TableIntroduction

In a Relational Database, keys are used to uniquely identify records and establish relationships between tables. They help maintain data integrity, reduce duplication, and improve data retrieval. The main types of keys are Super Key, Candidate Key, Primary Key, Foreign Key, and Composite Key.

Types of Keys

1. Super Key

A Super Key is a set of one or more attributes that uniquely identifies a record in a table.

Example

Student(Student_ID, Name, Email)

- Student_ID
- Student_ID + Name
- Student_ID + Email

All of these are Super Keys.

2. Candidate Key

A Candidate Key is the minimum set of attributes that uniquely identifies a record.

Example

- Student_ID
- Email

Both can uniquely identify a student, so they are Candidate Keys.

3. Primary Key

A Primary Key is the candidate key selected to uniquely identify each record in a table.

Features

- Cannot contain NULL values.
- Must be unique.
- Only one primary key is allowed per table.

Example

Student_ID is the Primary Key.

4. Foreign Key

A Foreign Key is an attribute in one table that refers to the Primary Key of another table.

Example

Student Table

Student_ID Name

101 Rahul

Marks Table

Student_ID Marks

101 95

Here, Student_ID in the Marks table is the Foreign Key.

5. Composite Key

A Composite Key is a key formed by combining two or more attributes to uniquely identify a record.

Example

Enrollment Table

Student_ID Course_ID Grade

The combination of Student_ID + Course_ID forms the Composite Key.

Advantages of Keys

- **Uniquely identify records.**
 - **Maintain data integrity.**
 - **Prevent duplicate records.**
 - **Establish relationships between tables.**
 - **Improve data retrieval and database performance.**
-

Summary Table

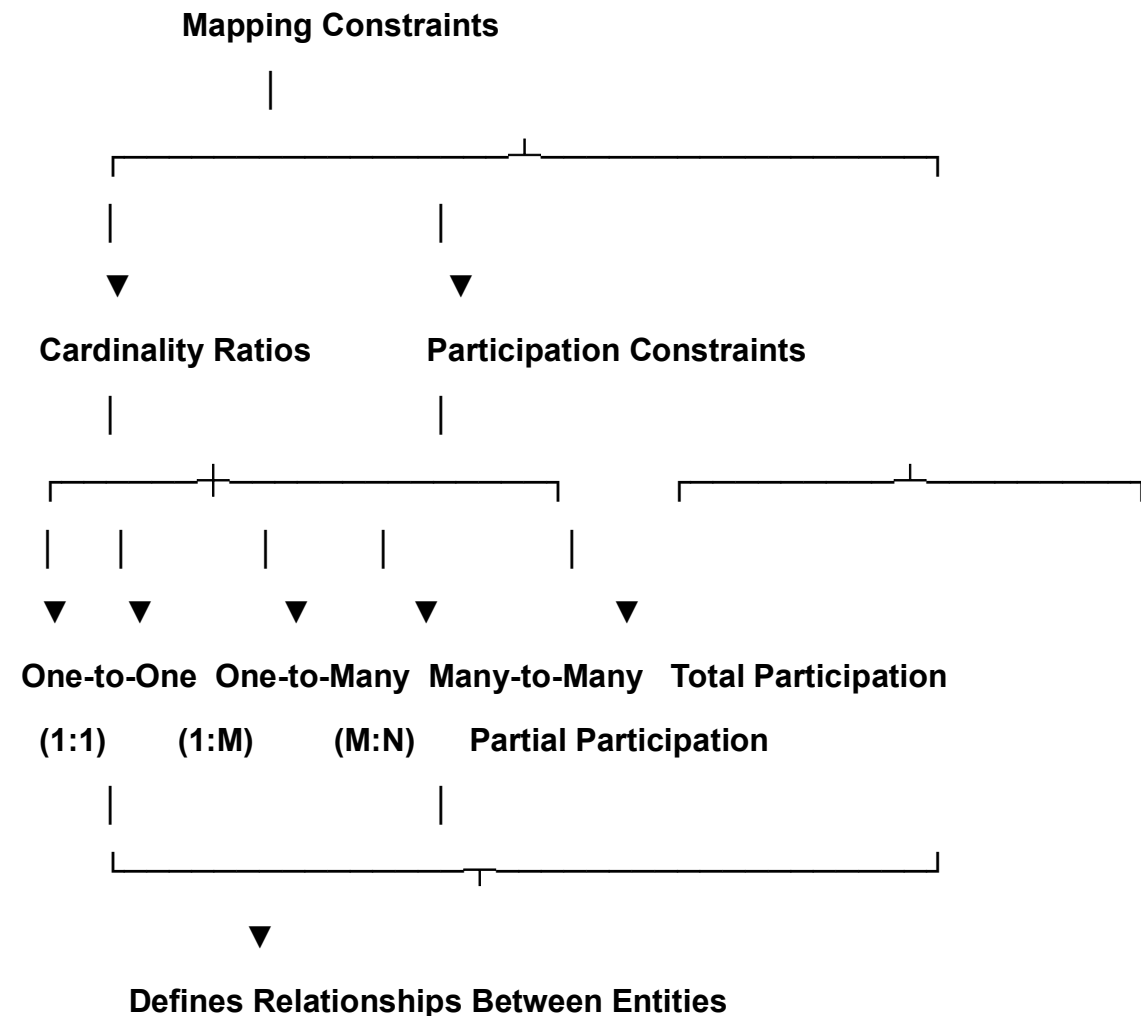
Key Type	Description	Example
Super Key	Uniquely identifies records	Student_ID, Student_ID + Name
Candidate Key	Minimal unique key	Student_ID, Email
Primary Key	Selected candidate key	Student_ID
Foreign Key	References another table's primary key	Student_ID in Marks table
Composite Key	Combination of multiple attributes	Student_ID + Course_ID

Conclusion

Keys play a vital role in a Relational Database Management System (RDBMS) by uniquely identifying records and maintaining relationships between tables. Super Keys, Candidate Keys, Primary Keys, Foreign Keys, and Composite Keys ensure data integrity, reduce redundancy, and improve database

efficiency. Understanding these keys is essential for designing well-structured and reliable databases.

Q7. Create a Concept Map Illustrating the Mapping Constraints in ER Models (Cardinality Ratios and Participation Constraints)



Introduction

In an Entity-Relationship (ER) Model, Mapping Constraints define how entities are related to each other. They specify the number of entities that can participate in a relationship (Cardinality Ratios) and whether participation is mandatory or optional (Participation Constraints). These constraints help in designing an accurate and efficient database.

1. Cardinality Ratios

Cardinality ratio specifies the number of instances of one entity that can be associated with instances of another entity.

Types of Cardinality Ratios

a) One-to-One (1:1)

One entity is associated with only one entity.

Example: One person has one passport.

b) One-to-Many (1:M)

One entity is associated with many entities.

Example: One department has many employees.

c) Many-to-Many (M:N)

Many entities are associated with many entities.

Example: Many students enroll in many courses.

2. Participation Constraints

Participation constraints specify whether an entity must or may participate in a relationship.

a) Total Participation

- **Every entity must participate in the relationship.**
- **Represented by a double line in an ER diagram.**

Example: Every employee must belong to a department.

b) Partial Participation

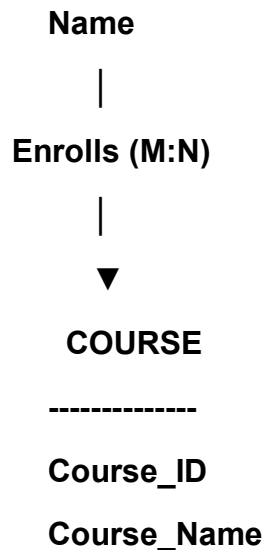
- **Participation is optional.**
- **Represented by a single line in an ER diagram.**

Example: A customer may or may not place an order.

Example ER Diagram

STUDENT

Student_ID



Here:

- **Cardinality: Many-to-Many (M:N)**
- **Participation: Can be Total or Partial depending on system requirements.**

Advantages of Mapping Constraints

- **Defines clear relationships between entities.**
- **Maintains data integrity.**
- **Reduces data redundancy.**
- **Improves database design.**
- **Makes database implementation easier.**

Summary Table

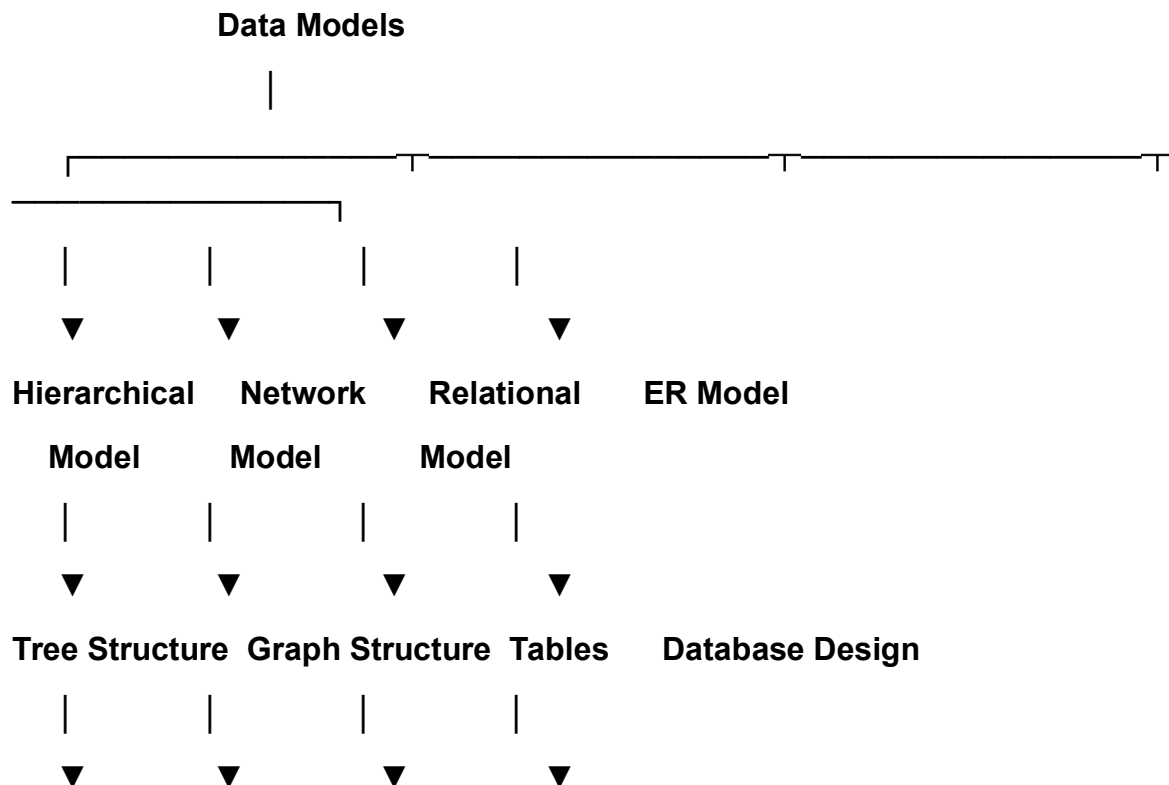
Constraint	Types	Example
Cardinality Ratio	1:1, 1:M, M:N	Student–Course
Participation Constraint	Total, Partial	Employee–Department

Conclusion

Mapping Constraints are an important part of the ER Model. They include Cardinality Ratios (One-to-One, One-to-Many, Many-to-Many) and Participation Constraints (Total and Partial Participation). These constraints define how

entities interact with each other, ensuring accurate database design, maintaining data integrity, and supporting efficient database management.

Q8. Map the Data Models Discussed in CO1 to Real-World Applications and Explain the Suitability of Each Model.



File Systems Airline System Banking System College/Hospital

A Data Model defines how data is stored, organized, and related in a database. Different data models are suitable for different types of applications based on their structure and relationships. The commonly used data models are Hierarchical, Network, Relational, and Entity-Relationship (ER) Models.

1. Hierarchical Data Model

The Hierarchical Model organizes data in a tree structure, where each child has only one parent.

Real-World Application

- File Management System
- Organization Chart

Suitability

- Suitable for one-to-many relationships.

- Simple and easy to understand.
 - Fast data retrieval for hierarchical data.
-

2. Network Data Model

The Network Model organizes data as a graph structure, allowing one record to have multiple parent records.

Real-World Application

- Airline Reservation System
- Telecommunications Network

Suitability

- Suitable for many-to-many relationships.
 - Supports complex data structures.
 - Flexible data access.
-

3. Relational Data Model

The Relational Model stores data in the form of tables (rows and columns).

Real-World Application

- Banking System
- College Management System
- Hospital Management System
- E-commerce Websites

Suitability

- Easy to use and maintain.
 - Supports SQL.
 - Reduces data redundancy.
 - Most widely used data model.
-

4. Entity-Relationship (ER) Model

The ER Model represents entities, attributes, and relationships graphically.

Real-World Application

- Database Design for Colleges
- Hospitals
- Libraries
- Online Shopping Systems

Suitability

- Simplifies database design.
- Clearly represents relationships.
- Easy to convert into relational tables.

Comparison Table

Data Model	Structure	Real-World Application	Suitability
Hierarchical Tree		File System, Organization Chart	Best for one-to-many relationships
Network	Graph	Airline Reservation	Best for many-to-many relationships
Relational	Tables	Banking, Hospital, College	Easy, flexible, and widely used
ER Model	Diagram	Database Design	Best for planning and designing databases

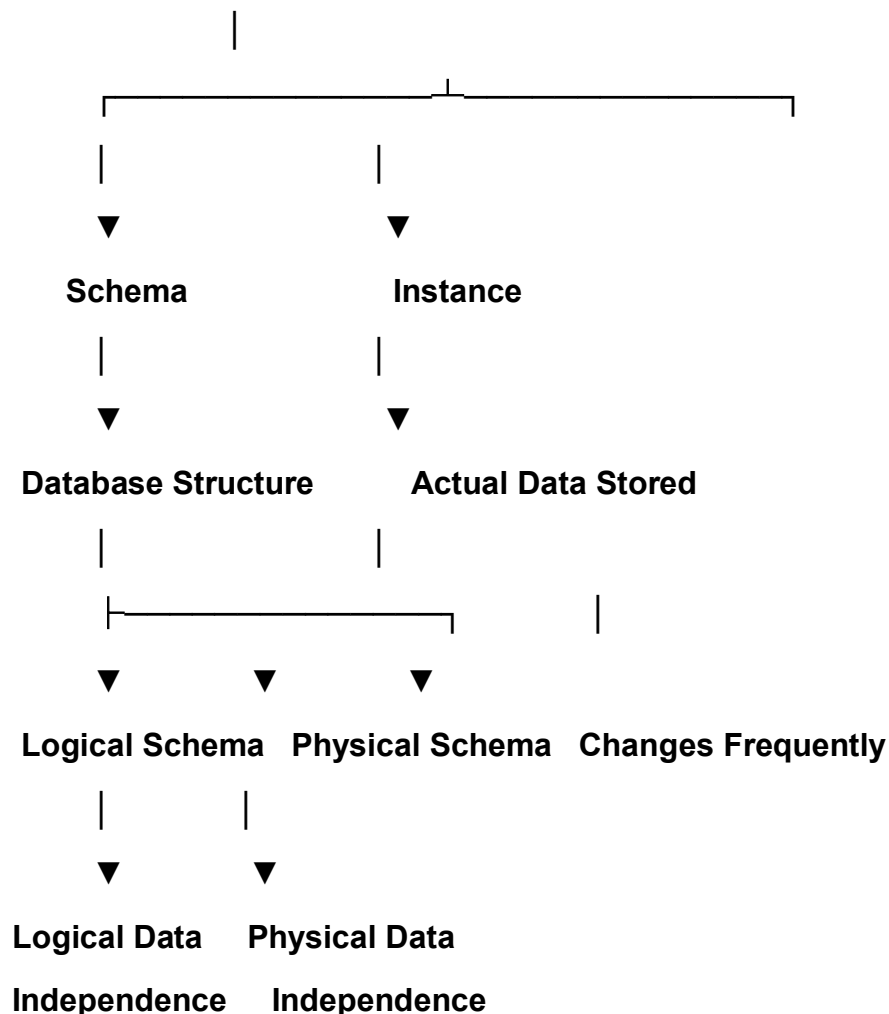
Advantages

- Organizes data efficiently.
- Supports different types of applications.
- Reduces data redundancy.
- Improves data consistency.
- Simplifies database design.
- Enables efficient data retrieval.

Conclusion

Different data models are suitable for different real-world applications. The Hierarchical Model is ideal for tree-structured data, the Network Model handles complex relationships, the Relational Model is the most widely used for modern databases, and the ER Model is mainly used for designing databases. Choosing the appropriate data model improves database performance, flexibility, and efficiency.

Q9. Construct a Concept Map for Schemas and Instances, Showing How They Relate to Physical and Logical Data Independence.



Introduction

In a Database Management System (DBMS), Schema and Instance are two important concepts. A Schema defines the structure of the database, while an Instance represents the actual data stored in the database at a particular time. These concepts are closely related to Logical Data Independence and Physical Data Independence, which make databases flexible and easy to maintain.

1. Schema

A Schema is the logical design or blueprint of a database. It defines:

- **Tables**
- **Attributes (Columns)**
- **Data Types**
- **Keys**
- **Relationships**
- **Constraints**

Example

Student Table Schema

Student_ID Name Department Age

Features

- **Created during database design.**
 - **Changes rarely.**
 - **Defines the database structure.**
-

2. Instance

An Instance is the actual data stored in the database at a particular moment.

Example

Student_ID Name Department Age

101 Rahul CSE 20

102 Priya AI 19

If a new student is added, only the instance changes, while the schema remains the same.

3. Logical Data Independence

Logical Data Independence is the ability to modify the logical schema without affecting the external views or application programs.

Example

- **Adding a new column Email to the Student table.**

- Existing applications continue to work without modification.

Advantages

- Easier database maintenance.
 - Supports database expansion.
 - No changes required in user applications.
-

4. Physical Data Independence

Physical Data Independence is the ability to modify the physical schema without affecting the logical schema.

Example

- Creating indexes.
- Changing file organization.
- Moving data to a new storage device.

Advantages

- Improves database performance.
 - Better storage management.
 - No changes required in application programs.
-

Relationship Between Schema, Instance, and Data Independence

- Schema defines the database structure.
 - Instance contains the actual data.
 - Logical Data Independence allows changes in the logical schema without affecting user views.
 - Physical Data Independence allows changes in physical storage without affecting the logical schema.
-

Summary Table

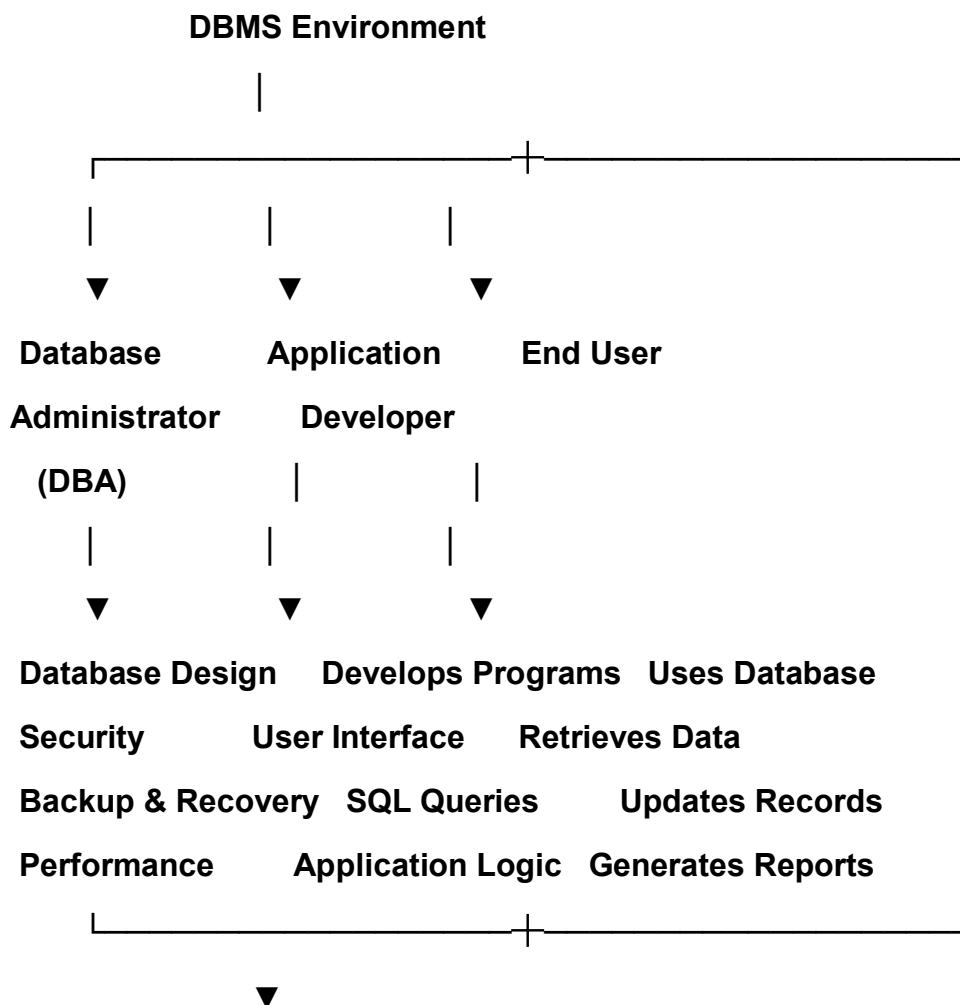
Concept	Description
Schema	Blueprint or structure of the database

Concept	Description
Instance	Actual data stored in the database
Logical Data Independence	Changes in logical schema do not affect user views
Physical Data Independence	Changes in physical storage do not affect logical schema

Conclusion

Schema and Instance are fundamental concepts in DBMS. The Schema defines the database structure, while the Instance represents the current data stored in the database. Logical Data Independence and Physical Data Independence make database systems flexible, efficient, and easy to maintain by allowing changes at one level without affecting other levels.

Q10. Design a Concept Map for the Roles and Responsibilities in a DBMS Environment: DBA, End User, and Application Developer.



Efficient Database Management

Introduction

A Database Management System (DBMS) involves different users who perform specific roles to manage and use the database effectively. The three main roles are the Database Administrator (DBA), Application Developer, and End User. Each role has different responsibilities that help ensure efficient, secure, and reliable database operations.

1. Database Administrator (DBA)

A Database Administrator (DBA) is responsible for managing and maintaining the database.

Responsibilities

- **Creates and maintains the database.**
 - **Controls user access and security.**
 - **Performs backup and recovery.**
 - **Monitors database performance.**
 - **Manages storage and data integrity.**
-

2. Application Developer

An Application Developer designs and develops applications that interact with the database.

Responsibilities

- **Develops database applications.**
 - **Writes SQL queries.**
 - **Designs user interfaces.**
 - **Tests and maintains applications.**
 - **Connects applications with the database.**
-

3. End User

An End User is the person who uses the application to access and manage data.

Responsibilities

- Retrieves information from the database.
 - Inserts, updates, and deletes data.
 - Generates reports.
 - Uses forms and applications to perform daily tasks.
-

Relationship Among the Roles

- The DBA manages and secures the database.
- The Application Developer creates software that communicates with the database.
- The End User uses the application developed by the developer to access and manipulate data.

Together, these roles ensure smooth and efficient database operations.

Summary Table

Role	Responsibilities
Database Administrator (DBA)	Database creation, security, backup, recovery, performance tuning
Application Developer	Develops applications, writes SQL queries, designs interfaces, testing
End User	Uses applications, retrieves data, updates records, generates reports

Advantages

- Clear division of responsibilities.
 - Improves database security.
 - Ensures efficient database management.
 - Supports smooth application development.
 - Enhances data accuracy and reliability.
-

Conclusion

In a DBMS environment, the Database Administrator (DBA), Application Developer, and End User play important roles. The DBA manages and secures the database, the Application Developer builds database applications, and the End User interacts with the database through these applications. Together, they ensure that the database system is secure, efficient, and easy to use.